

TP n°4 : Fichiers et Sérialisation en Java

Ressource R3.04 – Qualité de Développement • Documentation & Synthèse Pédagogique

Module : R3.04 Qualité de développement (Java)

Enseignant : Sébastien Thon

Niveau : BUT Informatique – Semestre 3

Conformité : 100% conforme au cours officiel (Chapitres 1 à 10)

1. Introduction & Objectifs Pédagogiques

Le TP n°4 aborde un cap fondamental de la programmation en Java : **l'interaction avec le système de fichiers et la persistance des objets** (sauvegarder l'état des données entre deux exécutions du programme). Ce travail pratique est scindé en deux exercices complémentaires :

- **Exercice 1 (Exploration et métadonnées de fichiers) :** Apprendre à inspecter un répertoire passé en argument de ligne de commande (`args[0]`), instancier un objet `java.io.File`, lister son contenu via `list()`, identifier les sous-dossiers et fichiers via `isDirectory()`, mesurer les tailles en octets avec `length()`, et réaliser un tri alphabétique insensible à la casse avec `Arrays.sort(tableau, String.CASE_INSENSITIVE_ORDER)`.
- **Exercice 2 (Gestion bancaire et persistance par sérialisation) :** Concevoir un système orienté objet composé d'une classe `Compte` et d'une classe `Banque` (encapsulant une `ArrayList<Compte>`). Rendre ces classes sérialisables via l'interface `java.io.Serializable` afin d'écrire et de relire l'état complet de la banque dans un fichier binaire avec `ObjectOutputStream` et `ObjectInputStream`, tout en sécurisant les opérations grâce à `try / catch` (gestion des exceptions `IOException` et `ClassNotFoundException`).

2. Tableau de Concordance : Syntaxe Utilisée vs Références du Cours

Toutes les notions, classes et méthodes employées dans cette réalisation sont **strictement issues du cours magistral** dispensé par M. Sébastien Thon. Le tableau ci-dessous en atteste la traçabilité intégrale :

Concept / Méthode	Référence Cours M. Thon	Fichier(s) du TP	Pourquoi cette syntaxe ? (Justification Pédagogique)
<code>args[0]</code>	Chapitre 10 (slide 3) & Chapitre 2	<code>ListerRepertoire.java</code>	Permet de récupérer les paramètres transmis par l'utilisateur lors du lancement en console (<code>java ListerRepertoire <dossier></code>).
<code>File(chemin)</code>	Chapitre 9, Slide 17 & 21	<code>ListerRepertoire.java</code> , <code>Banque.java</code>	Lie un chemin physique du système d'exploitation à un objet logique manipulable en Java.
<code>exists()</code>	Chapitre 9, Slide 18	<code>ListerRepertoire.java</code> , <code>Banque.java</code>	Vérifie si le fichier ou dossier existe physiquement avant toute opération, évitant les erreurs de fichier introuvable.
<code>length()</code>	Chapitre 9, Slide 19	<code>ListerRepertoire.java</code>	Fournit la taille exacte d'un fichier en octets sous la forme d'un entier <code>long</code> .
<code>list()</code>	Chapitre 9, Slide 21	<code>ListerRepertoire.java</code>	Retourne un tableau de chaînes (<code>String[]</code>) contenant le nom de tous les éléments du dossier.
<code>isDirectory()</code>	Énoncé TP4 (spécification)	<code>ListerRepertoire.java</code>	Méthode booléenne de <code>File</code> pour distinguer les répertoires des fichiers réguliers.
<code>Arrays.sort(...)</code>	Énoncé TP4 & Chapitre 5	<code>ListerRepertoire.java</code>	Trie le tableau en ordre alphabétique croissant insensible à la casse avec

Concept / Méthode	Référence Cours M. Thon	Fichier(s) du TP	Pourquoi cette syntaxe ? (Justification Pédagogique)
			<code>String.CASE_INSENSITIVE_ORDER</code> .
<code>implements Serializable</code>	Chapitre 9, Slide 24-25	<code>Compte.java</code> , <code>Banque.java</code>	Marqueur permettant au mécanisme de sérialisation Java d'enregistrer et reconstruire l'état des variables d'instance.
<code>ArrayList<Compte></code>	Chapitre 6, Slide 2 & 7-10	<code>Banque.java</code>	Collection dynamique permettant d'ajouter des comptes sans dimensionnement statique préalable.
<code>ObjectOutputStream</code>	Chapitre 9, Slide 23, 27-28	<code>Banque.java</code>	Flux de sortie permettant d'écrire directement un graphe d'objets dans un fichier binaire avec <code>writeObject()</code> .
<code>ObjectInputStream</code>	Chapitre 9, Slide 23, 27-29	<code>Banque.java</code>	Flux d'entrée permettant de désérialiser et reconstruire un objet Java depuis un fichier binaire avec <code>readObject()</code> .
<code>try / catch (IOException)</code>	Chapitre 10, Slide 4-5 & 14-17	<code>Banque.java</code> , <code>Principal.java</code>	Capture obligatoire des exceptions surveillées (Checked Exceptions) émises par les opérations d'entrées/sorties.
<code>Scanner(System.in)</code>	Chapitre 2, Slide 22-25	<code>Principal.java</code>	Lecture interactive des choix et des données saisies par l'utilisateur au clavier dans le terminal.

3. Architecture Globale de la Solution

Diagramme des Classes et Relations (UML)

```

+-----+
|                                     | (Exercice 1)
|                                     |
| + main(args: String[]): void      |
|                                     |
+-----+
| utilise                             |
| v                                   |
+-----+
|                                     |
| java.io.File & java.util.Arrays    |
|                                     |
+-----+

=====

+-----+
|                                     | (Exercice 2 - Q1)
|                                     |
| <<interface>> Serializable         |
|                                     |
| - serialVersionUID: long = 1L      |
| - proprietaire: String             |
| - solde: double                   |
|                                     |
+-----+
| + Compte(proprietaire: String, solde: double) |
| + getProprietaire(): String        |
| + setProprietaire(p: String): void |
| + getSolde(): double               |
| + setSolde(s: double): void        |
| + afficher(): void                 |
| + toString(): String               |
+-----+
|                                     |
| ^                                   |
| | 0..* est contenu dans           |
| |                                   |
+-----+
|                                     | (Exercice 2 - Q2)
|                                     |
| <<interface>> Serializable         |
|                                     |
+-----+

```

```

| - serialVersionUID: long = 1L
| - listeComptes: ArrayList<Compte>
+-----+
| + Banque()
| + ajouterCompte(c: Compte): void
| + chercherCompte(nom: String): Compte
| + afficherComptes(): void
| + afficherCompteClient(nom: String): void
| + sauvegarder(nomFichier: String): boolean
| + charger(nomFichier: String): Banque [static]
+-----+
|
| ^
| | instancie et pilote
| |
+-----+
| Principal (Exercice 2 - Q3)
+-----+
| + main(args: String[]): void
+-----+

```

4. Analyse Détaillée Fichier par Fichier et Méthode par Méthode

4.1 Exercice 1 : ListerRepertoire.java

Références de cours : Chapitre 9 (Slide 17, 18, 19, 21), Chapitre 5 (Tableaux), Chapitre 10 (Exceptions d'indices).

L'objectif de cette classe est d'agir comme une commande système équivalente à `ls -l` ou `dir`. Le programme s'articule autour des étapes logiques suivantes :

- Validation des arguments :** Si `args.length == 0`, l'utilisateur a oublié le répertoire. On affiche un guide d'utilisation (usage) et on quitte proprement sans lever de `ArrayIndexOutOfBoundsException` (Chapitre 10, slide 7).
- Instanciation logique de la ressource :** `File repertoire = new File(args[0]);` (Chapitre 9, slide 21).
- Contrôles physiques :**
 - `repertoire.exists()` : s'assure que le dossier est physiquement présent sur le disque (Chapitre 9, slide 18).
 - `repertoire.isDirectory()` : s'assure qu'il s'agit bien d'un dossier et non d'un simple fichier.
- Extraction et tri des noms :** `String[] listeFichiers = repertoire.list();` extrait les noms (Chapitre 9, slide 21). Puis `Arrays.sort(listeFichiers, String.CASE_INSENSITIVE_ORDER);` ordonne le tableau sans tenir compte de la casse (Énoncé TP4).
- Parcours et bilan statistique :** Une boucle `for` classique (Chapitre 5, slide 5) inspecte chaque élément via un sous-objet `new File(repertoire, nom)` pour afficher son type ([Répertoire] ou Fichier), mesurer sa taille via `element.length()` (Chapitre 9, slide 19) et cumuler la taille totale.

```

public class ListerRepertoire {
    public static void main(String[] args) {
        if (args.length == 0) {
            System.out.println("Erreur : Aucun répertoire spécifié.");
            return;
        }

        File repertoire = new File(args[0]);
        if (!repertoire.exists() || !repertoire.isDirectory()) {
            System.out.println("Erreur : Chemin invalide ou inexistant.");
            return;
        }

        String[] listeFichiers = repertoire.list();
        Arrays.sort(listeFichiers, String.CASE_INSENSITIVE_ORDER);

        // Bilan et affichage formaté...
    }
}

```

4.2 Exercice 2 (Q1) : Compte.java

Références de cours : Chapitre 3 (Slide 2, 3, 8, 10 - Encapsulation, Constructeur, Getters/Setters), Chapitre 9 (Slide 24-25 - Serializable).

La classe `Compte` modélise l'entité atomique du domaine bancaire :

- **Encapsulation :** Les variables d'instance `proprietaire` (String) et `solde` (double) sont strictement déclarées `private` (Chapitre 3, slide 2).
- **Interface `Serializable` :** En ajoutant `implements Serializable` (du package `java.io`), nous autorisons la machine virtuelle Java à convertir l'état complet d'un compte sous forme d'un flux d'octets sans avoir à coder d'écriture manuelle champ par champ (Chapitre 9, slide 24-25).
- **Constructeur & Méthodes :** Fournit un constructeur à deux paramètres, les accesseurs/mutateurs conformes, ainsi qu'une méthode `afficher()` (Chapitre 3, slide 10).

```
import java.io.Serializable;

public class Compte implements Serializable {
    private static final long serialVersionUID = 1L;
    private String proprietaire;
    private double solde;

    public Compte(String proprietaire, double solde) {
        this.proprietaire = proprietaire;
        this.solde = solde;
    }

    public String getProprietaire() { return this.proprietaire; }
    public void setProprietaire(String proprietaire) { this.proprietaire = proprietaire; }
    public double getSolde() { return this.solde; }
    public void setSolde(double solde) { this.solde = solde; }

    public void afficher() {
        System.out.println("Titulaire : " + this.proprietaire + " | Solde : " + this.solde + " €");
    }
}
```

4.3 Exercice 2 (Q2) : Banque.java

Références de cours : Chapitre 6 (Slide 2, 7-10 - ArrayList), Chapitre 9 (Slide 23-29 - Sérialisation d'objets composés), Chapitre 10 (Slide 5-8, 14-17 - Gestion des exceptions try/catch).

La classe `Banque` joue le rôle de conteneur d'agrégation :

- **Gestion dynamique des comptes :** Elle utilise un `ArrayList<Compte>` (Chapitre 6), initialisé dans le constructeur via `new ArrayList<Compte>()`.
- **Sérialisation en cascade :** Parce que `Banque` et `Compte` implémentent tous deux `Serializable`, la sérialisation de l'objet `Banque` sérialise automatiquement toute la liste des comptes contenus à l'intérieur ! C'est exactement le cas d'école présenté dans le cours de M. Thon avec la classe `Classe_etudiants` (Chapitre 9, slide 26).
- **Méthode `sauvegarder(String nomFichier)` :** Utilise `ObjectOutputStream` enrobant un `FileOutputStream` (Chapitre 9, slide 28). L'appel `fwo.writeObject(this);` écrit l'objet entier. En cas de problème disque, une `IOException` est capturée par le bloc `try / catch` (Chapitre 10, slide 16).
- **Méthode `charger(String nomFichier)` :** Vérifie d'abord `fichier.exists()` (Chapitre 9, slide 18). Si oui, ouvre un `ObjectInputStream(new FileInputStream(f))` (Chapitre 9, slide 29), désérialise avec `readObject()`, et transtype en `(Banque)`. Les exceptions `IOException` et `ClassNotFoundException` sont gérées proprement.

```
public boolean sauvegarder(String nomFichier) {
    try {
        ObjectOutputStream fwo = new ObjectOutputStream(new FileOutputStream(nomFichier));
        fwo.writeObject(this);
        fwo.close();
        return true;
    } catch (IOException e) {
```

```

        System.out.println("Erreur lors de la sauvegarde : " + e.getMessage());
        return false;
    }
}

public static Banque charger(String nomFichier) {
    File f = new File(nomFichier);
    if (!f.exists()) {
        System.out.println("Erreur : Le fichier \"" + nomFichier + "\" n'existe pas.");
        return null;
    }
    try {
        ObjectInputStream fRo = new ObjectInputStream(new FileInputStream(f));
        Banque b = (Banque) fRo.readObject();
        fRo.close();
        return b;
    } catch (IOException | ClassNotFoundException e) {
        System.out.println("Erreur chargement : " + e.getMessage());
        return null;
    }
}
}

```

4.4 Exercice 2 (Q3) : Principal.java

Références de cours : Chapitre 2 (Slide 18-20, 22-25 - Menu do/while, switch, Scanner).

La classe `Principal` contient le point d'entrée exécutable `main`. Elle gère le cycle de vie de l'application via un menu textuel interactif :

- **Boucle interactive :** `do { ... } while (choix != 6);` garantit l'affichage répété du menu jusqu'à ce que l'utilisateur choisisse explicitement de quitter.
- **Robustesse de la saisie utilisateur :** La saisie du montant utilise `Double.parseDouble(...)` (Chapitre 7, slide 1) entourée d'un `try / catch (NumberFormatException)` (Chapitre 10) et accepte aussi bien le point que la virgule décimale, évitant ainsi les plantages de saisie console.
- **Test de persistance réel :** L'utilisateur peut créer des comptes, les sauvegarder dans un fichier (ex: `banque.dat`), quitter l'application, la relancer et recharger ce même fichier pour constater la restauration intégrale des données.

5. Validation et Déroulement des Tests

5.1 Validation de l'Exercice 1 (ListerRepertoire)

Test exécuté sur le répertoire du projet :

```

$ java ListerRepertoire .
Contenu du répertoire : /home/nene_goto/Documents/REPO_COURS_BUT2_ARLES/JAVA_cours_BUT2/.
-----
Nom                                | Type          | Taille (octets)
-----
cours                             | [Répertoire] | -
Ouvrir_Netbeans_Debian.txt        | Fichier       | 83
TP02_01                           | [Répertoire] | -
TP03                              | [Répertoire] | -
TP04                              | [Répertoire] | -
-----
Statistiques finales :
- Nombre de fichiers   : 1
- Nombre de répertoires : 4
- Taille totale fichiers: 83 octets
-----

```

5.2 Validation de l'Exercice 2 (Cycle complet : Ajout, Sauvegarde, Extinction, Rechargement)

```

$ java Principal

=====
          GESTION DE LA BANQUE - MENU
=====

```

```

1. Ajouter un nouveau compte
2. Afficher le compte d'un client (par son nom)
3. Afficher l'ensemble des comptes
4. Sauvegarder les comptes dans un fichier
5. Charger les comptes depuis un fichier
6. Quitter
=====
Votre choix : 1

--- Ajout d'un compte ---
Nom du propriétaire : Alice
Solde initial (€) : 1500.50
Compte de Alice ajouté avec succès !

Votre choix : 1

--- Ajout d'un compte ---
Nom du propriétaire : Bob
Solde initial (€) : 230.00
Compte de Bob ajouté avec succès !

Votre choix : 3

===== Liste de tous les comptes bancaires (2) =====
[1] Titulaire : Alice | Solde : 1500.5 €
[2] Titulaire : Bob | Solde : 230.0 €
=====

Votre choix : 4

--- Sauvegarde de la banque ---
Nom du fichier de sauvegarde (ex: banque.dat) : banque_test.dat
Sauvegarde réussie dans le fichier : banque_test.dat

Votre choix : 6
Fermeture de l'application. Au revoir !

-----
-- RELANCE DU PROGRAMME & CHARGEMENT POUR VALIDER LA PERSISTANCE --
-----
$ java Principal
Votre choix : 5

--- Chargement de la banque ---
Nom du fichier à charger (ex: banque.dat) : banque_test.dat
Chargement réussi depuis le fichier : banque_test.dat

Votre choix : 3

===== Liste de tous les comptes bancaires (2) =====
[1] Titulaire : Alice | Solde : 1500.5 €
[2] Titulaire : Bob | Solde : 230.0 €
=====
Votre choix : 6
Fermeture de l'application. Au revoir !

```

6. Bilan Pédagogique & Bonnes Pratiques de Qualité

Synthèse des acquis du TP n°4

- **Zéro code hors-programme** : Ni streams lambdas ni bibliothèques tierces. Toutes les fonctionnalités reposent sur les API standard du cours (`java.io.File`, `ObjectInputStream`, `ObjectOutputStream`, `ArrayList`).
- **Encapsulation respectée** : Données membres privées, constructeurs dédiés, getters et setters rigoureusement implémentés.
- **Sérialisation déclarative** : La mise en œuvre de `implements Serializable` démontre l'élégance de Java pour persister des graphes d'objets complexes sans aucun parseur manuel.
- **Programmation défensive** : Contrôles systématiques de l'existence des fichiers avant lecture (`exists()`) et capture explicite des exceptions `IOException` et `ClassNotFoundException` pour empêcher les arrêts inopinés

du programme.